DA16 Cleaning the raw export

Week 6, Messy data and two tables | Session 16 of 24 | Python for Data Analysis

Today's question. If we clean the raw export carefully, does it give back the true revenue?

Objective. By the end: you can turn a messy export into a clean table with a pipeline that reruns, and explain why its total is lower than the true total.

> **WHERE THIS IS USED**
> A raw export from a real system is rarely tidy. Analysts clean it with a function so the same steps run again on next month's file. Could you clean a file you have never seen?

Before this session: DA15 Exploring relationships: scatter plots, correlation and cause. If you cannot explain the last example, use its error card with the instructor first.

Words for today

Word	What it means	Think of it like this
parse	Read text and turn it into a proper date or number.	Reading a handwritten date and writing it as a real date.
normalise	Make the same thing look the same everywhere, such as one spelling and one case.	Writing every name with a capital first letter.
lookup	A dictionary that turns one value into another.	The price board.
exclude	Leave a row out and count it, because it cannot be repaired.	Setting aside the damaged goods and counting them.
residual	The difference that is left after you have done everything right.	The small change nobody can account for.
pipeline	A series of cleaning steps that always run in the same order.	A production line.

Block A. Dates and names

The raw export has two date formats in one column: 2026-09-01 and 01/09/2026. Parse each format on its own, and keep the date that worked. Names appear as juice, JUICE and " Juice ". Strip the spaces, fix the case and look each name up in the products table.

Predict. Predict how many dates cannot be parsed after both formats have been tried. Write your guess on paper first.

Your guess, written before you run it

```
import pandas as pd
raw = pd.read_csv("sunrise_sales_raw.csv")
work = raw.drop_duplicates().copy()
```

```
iso = pd.to_datetime(work["sale_date"], format="%Y-%m-%d", errors="coerce")
slash = pd.to_datetime(work["sale_date"], format="%d/%m/%Y", errors="coerce")
work["sale_date"] = iso.fillna(slash)
print(work["sale_date"].isna().sum())
print(work["sale_date"].min().date(), work["sale_date"].max().date())
products = pd.read_csv("sunrise_products.csv")
lookup = {n.title(): n for n in products["name"]}
work["product"] = work["product"].str.strip().str.title().map(lookup)
print(work["product"].isna().sum(), work["product"].nunique())
```

Run it, then compare. The cell prints 0 dates that cannot be parsed, then the range 2026-09-01 to 2026-10-30. The names step prints 0 names left unmatched and 15 different products. Every date and product name is now one consistent form.

Investigate.

1. Why parse the two formats separately and join the results?
2. What does errors="coerce" do?

Modify. Count how many rows used the slash format.

Block B. Units, blanks and the residual

Some units are written as words, such as two. A lookup turns them into numbers. Blank units cannot be repaired, so we exclude those rows and count them. Then we compare with the true total. The two totals will differ, and the difference has a name.

Predict. Predict whether the cleaned revenue will be above, equal to or below the true revenue of 6375900. Write your guess on paper first.

Your guess, written before you run it

```
words = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5}
work["units"] = pd.to_numeric(work["units"].replace(words), errors="coerce")
excluded = work[work["units"].isna()]
clean = work.dropna(subset=["units"]).copy()
clean["units"] = clean["units"].astype(int)
clean["line_value"] = clean["units"] * clean["unit_price_ugx"]
print(len(excluded), len(clean))
print(clean["line_value"].sum())
true_sales = pd.read_csv("sunrise_sales.csv")
true_total = (true_sales["units"] * true_sales["unit_price_ugx"]).sum()
residual = true_total - clean["line_value"].sum()
print(residual)
```

Run it, then compare. 40 rows are excluded and 1473 rows are clean. The cleaned revenue is 6250500, which is 125400 below the true 6375900. The residual of 125400 is the value of the 40 excluded sales. It is not a mistake in the pipeline. It is the cost of rows that cannot be recovered.

Investigate.

1. Why is the cleaned total below the true total?

2. Would it be honest to report that the two totals match?

Modify. Prove that the residual equals the value of the 40 excluded sales, using the true sales table.

Make

Put the cleaning steps in one function called clean_sales(raw) that returns the clean table and the excluded rows. Run it on the raw export and check with assert that the clean table has 1473 rows and the clean revenue is 6250500. Write one sentence naming the residual and its cause.

Save your work as DA16. Keep the worked example and put your own attempt below it. Write two sentences: what you changed and how you checked it.

Error card

You see	It means	Fix
ValueError: time data does not match format	A date is written in a form the format string does not describe.	Use errors="coerce" for one format and fill the gaps with the other format.
The clean table has 1538 rows, not 1513 or 1473	The duplicates were not removed before the other steps.	Call drop_duplicates() first and print len() after every step.

Exit ticket

A careful clean can still leave a gap, and I report the gap and its cause.

AI use follows AI_USE_POLICY.pdf. In weeks 1 to 6 no AI-generated solutions are used for exercises, labs or build tasks. Practise for 20 to 30 minutes between classes by repeating the Make task with one value changed. If you are stuck for 30 minutes, save the error and bring it to class. A file saved on your laptop has not been shared until you use the route in START_HERE.pdf.